Nandita Balasubramanian, Robin-Marcel Hanne, Thomas Wolf

End Project Report

A sophisticated agent model

Github Repository: https://github.com/togewolf/computer_games_end_project

What is the game?

The game is a 1-versus-1 wizard duel: the player faces an AI-controlled enemy wizard across a vertical arena, each standing on the battlements of an opposing castle tower. Both wizards attack by casting elemental projectiles; the first wizard whose health reaches zero loses the match.

Combat is built around an elemental counter system with four elements: water beats fire, fire beats nature, nature beats light, and light beats water. When two opposing projectiles collide, the winning element destroys the losing one, while equal elements annihilate each other. Reading the incoming spell and answering it with the correct counter is the core skill of the game – for the player and the agent alike.

Casting is limited by a regenerating mana pool. The mana cost of a spell scales with its cast mode (simple projectile or targeted cast) and its projectile speed, so stronger casts drain the budget faster. This prevents the duel from degenerating into button-spamming and forces economic decisions about when and how to cast. Projectiles can also be dodged: both wizards can move horizontally along their battlements.

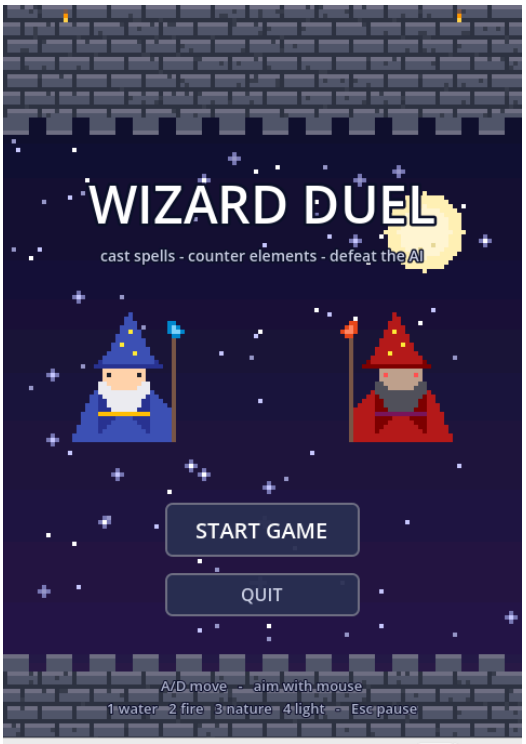
Spells can be cast in two ways: conventionally with the keyboard (keys 1–4 for the elements, mouse to aim, A/D to move, 6-8 to control speed, 9 and 0 to control the projectile mode (normal and homing)) or hands-free through the voice engine described below, where each spell component is bound to a spoken magic word. A complete game loop frames the duel: a main menu leads into the match, the game can be paused at any time with Esc, and a victory/defeat screen with a restart option ends it.

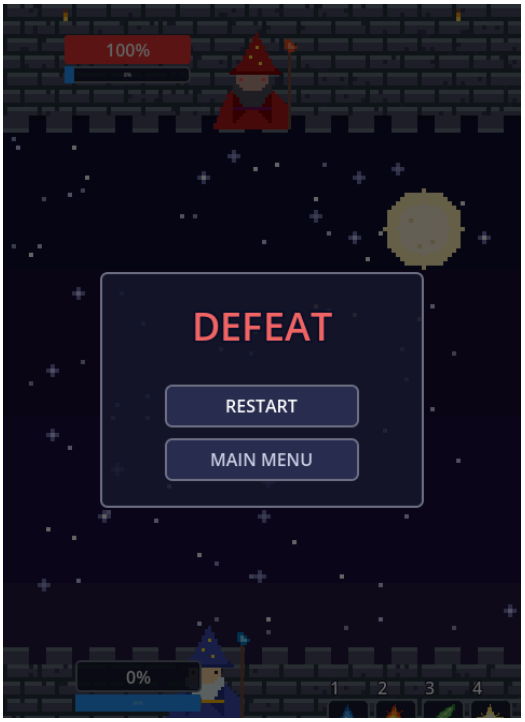
UI elements & functionality

All graphics are custom pixel art (a 480×720 night-sky arena with two opposing castle towers, two wizard sprites and four projectile sprites), rendered with nearest-neighbour filtering so the pixels stay crisp. Each element has a distinct projectile silhouette – flame, droplet, leaf and star – so an incoming spell can be recognized at a glance. This readability is deliberate: the counter system is only playable if a threat can be identified within a fraction of a second.

The interface consists of: a main menu (Start/Quit plus an overview of all controls); colour-coded health bars (green for the player, red for the agent) with blue mana bars beneath them, updated live through the wizards' health and mana signals; a spell hotbar showing each element's icon with its keyboard key; two sliders to adjust the agent's reaction speed and randomness during play, used for difficulty control and for demonstrating the

agent's behaviour; a pause menu (Esc freezes the entire match via the scene tree's pause mode); and a game over screen that halts the match at 0 HP and offers Restart or Main menu.





The Agent

The agent operates once every game tick. Every tick the agent can decide to cast a spell and update its plan and behaviour state.
Threats to the agent are spells cast by the player. Its first priority is always to assess the current playing field. This means that it gathers a list of all threats currently on the screen. For this, it scans the area in front of it for nodes in the “Projectiles” group and checks whether they belong to him or the player (enemy). In case of the latter it checks whether the threat has already been dealt with. For this, the agent keeps track of a list of threats it encountered but which are already being parried with a corresponding counter attack.
If the threat is not dealt with yet, it calculates

- the position the projectile can hit the agent in
- The distance of the projectile to the agent
- It’s estimated time of arrival until it would hit the agents' bounding box (declared as personal_eta. If the projectiles direction is not aiming for the agent’s current position this eta goes towards infinity)
- The estimated time of arrival until the projectile hits the agents y axis and thus crosses the moveable area (defined eta).

It then ranks all these threats in a sorted list based on their priority. Because projectiles can have different speeds, the threats are ranked based on their ETA rather than a simple distance check.
With the threats now ranked based on priority (ETA) the agent now resumes to make an informed decision on what to do. For every threat a decision must be made: Do we try to evade it, or try to parry it with a projectile of our own costing us mana we might need in order to attack the player.
To complicate the decision making even further, multiple projectiles can block many parts of the movement axis at the same time.
Thus, the agent now constructs a set of intervals, referred to as the “danger zones”. The concept of “danger zones” in this project can be seen as a specialized 1D Influence Map with a spatial representation. First, we need to know where on our movement (x-axis) this projectile will land. This is computed using simple trigonometry by solving for a scaling factor such that the projectile's direction vector scaled by said factor added on top of the projectile's current position lands on our agents y axis.
As a n example: Our agent is positioned on y = 400 and we can only move on the x-axis. The projectile now has a current position of (200, 300) and a (normalized, but for simplicity here this isn’t the case) direction vector of (0.25, 0.5). Our scaling factor is now (400 - 300) / 0.5 = 200 in this case, which allows us to then deduce where the projectile will land on the x axis at our y height at the given trajectory. We perform this calculation once on the y-coordinate of the bottom of our agents hitbox and once at the top. This gives us two impact points i_0 and i_1 which we sort using minimum and maximum to determine the left and right impact points. We now define a clearance (the distance our agents centre point needs to be away from these points in order to remain safe as:

$$c = r_{projectile} + r_{wizard} + p$$

Where $r_{projectile}$ is the radius of the projectile, r_{wizard} is the radius of our wizard and $p > 0$ is a padding added for extra safety.

For every threat we can thus define an interval on our x-axis on where the projectile would damage us. These danger zones are therefore defined as
$dz = (max(min(i_0, i_1) - c, a_{min}), min(max(i_0, i_1) + c) + a_{max})$

Because of the aforementioned padding p, we are always safe of this projectile at the boundaries of the interval.
We now check our current position and check whether we are safe at our spot by checking if it overlaps with any danger zones. If it does, we start checking for better spots.

For this, we consider every unique boundary of the danger zones a contender. Every contender that is not reachable before the time it takes the projectile with the shortest ETA to cross our y axis is filtered out. Every other contestant is then iterated over each danger zone and ranked based on how many threats are still hitting.
We pick the contestant with the least bullets hitting it. If there are still bullets hitting the best reachable contestant we sort them and mark our tick's spell cast intent to parry the closest once it is close enough.

Once the agent deems itself to be safe, and have no intent of parrying a projectile it considers its offensive options. For this the agent first determines his "free cashflow". As in, how much mana do I have to spare that I do not need if I suddenly needed to parry all spells?
Given this mana cashflow the, the agent first randomly picks an element and a projectile speed and mode. If the spell is castable with his budget he casts the spell. If the projectile speed and mode are outside its mana budget it decides to randomly either downgrade the projectile speed or its mode, based on a random value skewed based on the agents attack preference parameter.
If the agent can fire his degraded version now, he casts the spell. Otherwise, he repeats the downgrading process up to 5 times, before deciding not to cast.

Voice Engine

The game uses a "wakeword engine" to detect spoken keywords:

```
"igi": "fire",
"vova": "water",
"rube": "nature",
"pikat": "light",
"zollzag": "target",
"bumbo": "projectile",
"noxo": "slow",
"trodu": "fast",
"pringo": "fastest"
```

The training pipeline and training data were recycled from a different project and are not included in the repository.

After starting the python script which runs in parallel to the game (but is not necessary for the game to run, since keys assigned to the same effects can be used equivalently), the system continuously listens to the microphone. The captured audio is sliced into rolling

chunks (0.05 seconds each), which are then converted into a Mel-Spectrogram (visual heatmap of the audio's frequency bands over time). The model analyzes the incoming stream dynamically. To prevent misfiring, the algorithm evaluates only the "center" of a rolling 1.4-second window.
A real-time smoothing queue averages the predictions over the last few frames to filter out random noise spikes. The moment a spell (e.g. "igi") crosses the confidence threshold (set to 0.4 currently), the engine emits an UDP packet to Godot and the spell is cast.

The underlying neural network is a **Convolutional Recurrent Neural Network (CRNN)**. It combines two powerful machine learning paradigms to process audio efficiently:

Convolutional Layers (CNN): A 2D Convolutional network scans the Mel-spectrogram to extract low-level acoustic features, such as pitch changes, formants, and phoneme shapes. It only pools along the frequency dimension, preserving the exact temporal resolution of the audio.
Recurrent Layers (Bi-GRU): The extracted features are flattened and fed into a Bidirectional Gated Recurrent Unit. The RNN tracks the chronological sequence of the sounds, allowing the model to understand how syllables flow together over time.
Dense Output: Finally, a Softmax classification layer outputs the probabilities for the 9 active spells, plus "noise" and "unknown".

Traditional CNNs summarize an entire audio file into a single prediction, meaning you must finish recording the whole sound snippet before the AI can analyze it. This creates an unavoidable, immersion-breaking delay in gameplay.

By utilizing a CRNN, the network feeds patches of the sequence frame-by-frame into the recurrent layers. The GRUs maintain an ongoing "memory" of the audio state. Because the recurrent layer outputs predictions dynamically at every single time step, the model is highly responsive. This allows the game to cast the spell the exact moment the player finishes pronouncing the magic word.

References

The Agent AI is a highly specialized design for the given game. It builds on broad concepts in Game AI development such as a

- Behavioural State Machine and a Behaviour Tree
  https://www.gameaipro.com/GameAIPro/GameAIPro_Chapter10_Building_Utility_Decisions_into_Your_Existing_Behavior_Tree.pdf
- Reynolds, Craig W. "Steering behaviors for autonomous characters." *Game developers conference*. Vol. 1999. 1999.
- and Modular Tactical Influence Maps
  https://www.gameaipro.com/GameAIPro2/GameAIPro2_Chapter30_Modular_Tactical_Influence_Maps.pdf

- The architecture of the voice engine is heavily inspired by the methodologies detailed in: *"Convolutional RNN: an Enhanced Model for Extracting Features from Sequential Data"* (Keren & Schuller, 2016)